Unit 1 Lesson 3: Exploring Data Types

Practice

Name:	**Exploring Data Types**

🎯 **Objective:** **I can use and describe Python's data types — strings, integers, floats, and booleans — and show how they are used in a real-world program.**

✏️ **Practice:**

Starter Skills:

Create and save a new Python file called **03_Practice.py** inside of your desktop **Game Programming I > Unit 1 > L3 Exploring Data Types** folder.

1. Create one variable of each main data type:
    a. Your name (string)
    b. Your age (integer)
    c. Your height in meters (float)
    Use this feet to meters conversion tool if needed
    d. Whether you enjoy being outdoors (boolean)

    Paste a screenshot of your code below:

2. Use `print()` to display all your variables on one line using **commas**.
    Paste a screenshot of your code below:

3. Use type() to print the data type of each variable below your summary.
    Example:

    ```
    type(age)
    ```

    Paste a screenshot of your code below:

4. Follow the instructions carefully:
    a. Create these variables:

    ```
    school = "Von Steuben"
    students = 1200
    gpa = 3.5
    in_session = True
    ```

    b. Use `print()` and `type()` to display both the value and its data type for each of your variables. Remember, if you use concatenation, there is an extra step required to make the message display our information. Use your notes as needed!
    Example:

    ```
    School: Von Steuben | Type: <class 'str'>
    ```

5. Find the error(s) in the code below. Correct any errors then paste your screenshot below.

```
# Debug this program
students = "25"
print("There are" + students + "students in class.")
print(type(students))
```

6. Let's practice converting between data types and checking if it worked.
   a. Copy this code:

```
miles = "5"
hours = "2.5"
print(type(miles), type(hours))
```

   b. Convert both variables to numbers and print their types again. Make sure to convert to the correct type– be careful of decimal values versus whole numbers.
   Paste your code below:

   c. Calculate and print **speed = miles / hours**, then print its type.
   Paste your code below:

7. Follow the instructions below.
   a. Copy this working example exactly as it appears:

```
grade_level = 11
student_name = "Alex"
print("Name:", student_name, "| Grade:", grade_level)
```

   b. Add your own variable called `average_score` (as a float).
      - Use concatenation (`+`) and `str()` to print all three values together.
      - Then print their data types on separate lines.
      - **Challenge**: Create a boolean variable like passed_exam and print that too!
      Paste your code:

8. Now let's test what happens when you convert between different types.
    a. Copy this code and predict what you think each line will print:

```
num = 10
text = "7"
print(str(num) + text)      # Prediction:
print(num + int(text))      # Prediction:
print(float(num) + 0.5)     # Prediction:
```

    b. Run it and write short notes in comments for what you learned. Paste your code below:

Power-Up Skills:

9. You are going to write some code to plan a camping trip.
    a. Create variables to represent:
        - Destination
        - Number of friends going
        - Average cost per person
        - Is the trip confirmed?
    b. Print a short summary using concatenation:
    c. Use `type()` to check your variable types and explain your result in a comment.
    Paste your code below:

Legendary Skills:

10. You will be writing some code to help plan your friend's next outdoor adventure. Your code should:
    a. Ask the user for input using `input()` for:
        - Activity name
        - Hours planned (convert to a float)
        - Number of friends joining (convert to an int)

```
variable_name = input("Prompt or question shown to the user: ")
```

```
name = input("What is your name? ")
print("Hi", name, "welcome to class!")
```

    b. Create one boolean variable called `is_weather_good` and set it to `True` or `False`.
    c. Use `type()` to show the type of each variable.
    d. Print a summary using commas or concatenation.
    Paste code screenshot:

11. Add a simple **if/else statement** to your code from the previous question. You'll check whether `is_weather_good` is `True` or `False`, and print a message based on that condition.

```
# Example structure only:
if condition_is_true:
    print("Something happens if the condition is true.")
else:
    print("Something happens if the condition is false.")
```

```
is_sunny = True

if is_sunny:
    print("Perfect day for outdoor practice!")
else:
    print("Looks like an indoor day instead.")
```

Paste code below:

🏆 **Criteria for Mastery:**

This rubric shows you exactly what strong work looks like on this Mastery Check. Use it to understand the skills you need to show, how your work will be scored, and what to focus on as you practice. The goal is to help you see where you are now and what steps will help you grow.

0- Not Evident	There is no evidence of understanding. Work may be missing, blank, incorrect, or not submitted. Student does not identify any data types correctly. No attempt to explain errors, conversions, or type() output.
1- Insufficient	Work is attempted, but demonstrates little to no understanding of data types. Major errors prevent the code from working or showing correct results. Student cannot clearly identify basic types (string, int, float, boolean). Error explanations are unclear or missing.
2- Beginning Proficiency	Student shows partial understanding, but work is inconsistent. Many data type identifications are incorrect or confused. Conversion steps (str, int, float) are attempted but incomplete or incorrect. Student may know what type() does but cannot apply it reliably. Debugging attempts are present but unfinished or show misunderstandings.
3- Nearing Proficient	Student shows a general understanding with only small mistakes. Most data types are identified correctly. type() is used correctly in most cases. Some errors in explanations or missing details. Type conversion may be slightly incorrect (ex: wrong target type). Debugging is mostly correct but may miss one small issue.

4- Proficient	Student meets expectations and shows a clear, correct understanding of data types. All data types are identified accurately. Correct use of type(), type conversion, and proper syntax. Debugging is correct and fixes the problem fully. Work is readable and logically organized.
5- Mastery	Student exceeds expectations and demonstrates full confidence and accuracy. All work is correct, clearly explained, and well-organized. Data types, type(), and conversions are used correctly in every case. Student uses clean, professional Python formatting throughout.

Mastered?: ✅ **Yes** (advance to next lesson) / ↩️ **No** (revise practice work, then reassess)